

Cahier des charges - ArtSphere

1. Présentation du projet

1.1 Contexte

ArtSphere est une plateforme web conçue pour les artistes et les amateurs d'art, leur permettant de créer, partager, et collaborer sur des dessins numériques. L'objectif est de fournir un espace interactif où les utilisateurs peuvent exprimer leur créativité, découvrir les œuvres d'autres artistes, et interagir à travers des fonctionnalités sociales comme les likes, les commentaires, et un système de suivi.

1.2 Objectifs

- Offrir un éditeur de dessin intuitif avec des outils de base (pinceau, undo/redo, effacer le canevas).
- Permettre aux utilisateurs de partager leurs dessins dans une galerie publique.
- Intégrer des fonctionnalités sociales : aimer, commenter, suivre d'autres utilisateurs.
- Fournir un espace de profil personnalisable (nom d'utilisateur, photo de profil, bio).
- Garantir une expérience utilisateur fluide et responsive sur desktop et mobile.
- Assurer la sécurité des données et des interactions via une authentification robuste.

1.3 Public cible

- **Artistes amateurs et professionnels** : Pour créer et partager leurs dessins.
- **Amateurs d'art** : Pour découvrir et interagir avec les œuvres d'autres utilisateurs.
- **Utilisateurs de tous âges** : Avec une interface simple et intuitive.

2. Spécifications fonctionnelles

2.1 Fonctionnalités principales

2.1.1 Authentification

- Connexion : Les utilisateurs peuvent se connecter via Google ou email/mot de passe.
- Inscription : Création de compte avec email/mot de passe.

- Gestion du mot de passe : Possibilité de changer le mot de passe depuis la page de profil.
- Déconnexion : Option pour se déconnecter.

2.1.2 Éditeur de dessin

- Outils de dessin :
 - Pinceau avec tailles ajustables.
 - Undo/redo pour annuler ou refaire des actions.
 - Bouton pour effacer le canevas.
- Sauvegarde : Les dessins peuvent être sauvegardés dans la galerie avec un titre, une description, des tags, une catégorie, et une visibilité (publique/privée).

2.1.3 Galerie

- Affichage des dessins :
 - Liste des dessins publics (et privés pour l'utilisateur connecté).
 - Tri par popularité (nombre de likes) ou récence.
 - Filtrage par tags, catégories, ou utilisateur.
- Interactions :
 - Aimer un dessin (un utilisateur ne peut aimer qu'une fois).
 - Partager un dessin sur Twitter ou Facebook.
 - Voir les détails d'un dessin (titre, description, tags, catégorie, visibilité, nombre de likes, créateur).
- ****Pagination**** : Chargement progressif des dessins avec un système de défilement infini.

2.1.4 Profil utilisateur

- Personnalisation :
 - Nom d'utilisateur (modifiable).
 - Photo de profil (upload via Firebase Storage).
 - Bio (texte descriptif, max 500 caractères).
- Gestion des dessins :
 - Liste des dessins créés par l'utilisateur.

- Possibilité de supprimer un dessin.
- Système de suivi :
 - Suivre/désabonner d'autres utilisateurs.
 - Liste des abonnés et abonnements.

2.1.5 Interactions sociales

- Likes : Les utilisateurs connectés peuvent aimer les dessins.
- Commentaires : Les utilisateurs connectés peuvent commenter les dessins (fonctionnalité à implémenter).
- ****Suivi**** : Suivre d'autres utilisateurs pour voir leurs nouveaux dessins dans la galerie.

2.2 Fonctionnalités secondaires

- Design responsive : L'application doit être utilisable sur desktop, tablette, et mobile.
- Animations : Utilisation de Framer Motion pour des transitions fluides dans la galerie.
- Partage social : Boutons pour partager les dessins sur Twitter et Facebook.

3. Spécifications techniques

3.1 Architecture

- Frontend : React avec Vite pour un développement rapide et des builds optimisés.
- Backend : Firebase Functions pour les opérations serverless.
- Base de données : Firebase Firestore pour stocker les dessins, profils, likes, et commentaires.
- Stockage : Firebase Storage pour les photos de profil.
- Authentification : Firebase Authentication (Google et email/mot de passe).

3.2 Technologies utilisées

- React : Framework frontend pour l'interface utilisateur.
- Vite: Outil de build pour le développement et la production.
- Firebase :
 - Authentification : Gestion des utilisateurs.

- Firestore: Base de données NoSQL pour les données structurées.
- Storage : Stockage des fichiers (photos de profil).
- Functions : Backend serverless pour les opérations sécurisées.
- Tailwind CSS : Framework CSS pour un design responsive.
- Framer Motion: Animations fluides.
- React Slick : Carrousel pour la galerie (si utilisé dans d'autres parties de l'app).
- Lottie : Animations interactives (si utilisé dans d'autres parties de l'app).

3.3 Structure des données (Firestore)

Collection `users`

- `userId` (ID) : Identifiant unique de l'utilisateur (UID de Firebase Auth).
- `username` (string) : Nom d'utilisateur (unique).
- `email` (string) : Adresse email.
- `profilePicture` (string) : URL de la photo de profil (Firebase Storage).
- `bio` (string) : Bio de l'utilisateur (max 500 caractères).
- `followers` (array) : Liste des UUIDs des abonnés.
- `following` (array) : Liste des UUIDs des utilisateurs suivis.
- `createdAt` (timestamp) : Date de création du profil.

Collection `drawings`

- `drawingId` (ID) : Identifiant unique du dessin.
- `title` (string) : Titre du dessin.
- `description` (string) : Description du dessin.
- `image` (string) : URL de l'image du dessin (Firebase Storage).
- `tags` (array) : Liste de tags (ex. ["paysage", "abstrait"]).
- `category` (string) : Catégorie (ex. "Paysage", "Abstrait", "Other").
- `visibility` (string) : Visibilité ("public" ou "private").
- `userId` (string) : UID de l'utilisateur créateur.

- `createdAt` (timestamp) : Date de création.
- Sous-collection `likes` :
 - `liked` (ID) : UID de l'utilisateur qui a aimé.
 - `userId` (string) : UID de l'utilisateur qui a aimé.
 - `createdAt` (timestamp) : Date du like.

Collection `comments`

- `commentId` (ID) : Identifiant unique du commentaire.
- `drawingId` (string) : ID du dessin associé.
- `userId` (string) : UID de l'utilisateur qui a commenté.
- `content` (string) : Contenu du commentaire.
- `createdAt` (timestamp) : Date du commentaire.

3.4 Contraintes techniques

- Sécurité :
 - Toutes les opérations Firestore (lecture/écriture) doivent être effectuées via Firebase Functions pour contourner les restrictions de permissions.
 - Les utilisateurs non connectés ne peuvent pas aimer, commenter, ou suivre.
- Performance :
 - Chargement progressif des dessins dans la galerie (pagination ou défilement infini).
 - Optimisation des images (compression avant upload dans Firebase Storage).
- Compatibilité :
 - Navigateurs supportés : Chrome, Firefox, Safari, Edge (dernières versions).
 - Responsive design pour desktop, tablette, et mobile.

4. Exigences non fonctionnelles

4.1 Ergonomie

- Interface intuitive et facile à utiliser.

- Design moderne avec des animations fluides (via Framer Motion).
- Temps de chargement minimal pour les dessins et les profils.

4.2 Sécurité

- Authentification sécurisée via Firebase Authentication.
- Protection des données utilisateur (Firestore et Storage).
- Validation des entrées utilisateur (ex. longueur maximale de la bio, unicité des noms d'utilisateur).

4.3 Disponibilité

- L'application doit être disponible 24/7, sauf pendant les mises à jour planifiées.
- Déploiement sur Firebase Hosting pour une haute disponibilité.

5. Livrables

5.1 Livrables principaux

- Code source : Dépôt GitHub avec le code client (React) et serveur (Firebase Functions).
- Documentation :
 - README.md avec instructions d'installation, d'utilisation, et de déploiement.
 - Documentation technique (structure des données, API des fonctions).
- Application déployée : URL de production (ex. `https://artsphere-72332.web.app`).
- Captures d'écran : Screenshots des principales fonctionnalités (éditeur, galerie, profil).

5.2 Livrables secondaires

- Présentation : Diapositives pour la présentation au jury (introduction, fonctionnalités, défis, démo, perspectives).
- Vidéo de démonstration : Vidéo montrant l'utilisation de l'application (optionnel).

6. Planning et jalons

6.1 Phases du projet

1. Phase 1 : Analyse et conception (1 semaine)

- Rédaction du cahier des charges.
- Conception de l'architecture (Firestore, Firebase Functions).
- Maquettes de l'interface utilisateur (optionnel).

2. Phase 2 : Développement (4-6 semaines)

- Configuration de Firebase (Auth, Firestore, Storage, Functions).
- Développement de l'éditeur de dessin.
- Développement de la galerie.
- Développement des profils utilisateurs et du système de suivi.
- Intégration des fonctionnalités sociales (likes, commentaires).

3. Phase 3 : Tests et déploiement (1-2 semaines)

- Tests unitaires et fonctionnels.
- Tests de performance et de compatibilité.
- Déploiement sur Firebase Hosting et Functions.
- Corrections de bugs.

4. Phase 4 : Présentation (1 semaine)

- Préparation de la présentation (diapositives, démo).
- Rédaction de la documentation finale.

6.2 Jalons

- **Semaine 1** : Cahier des charges validé.
- **Semaine 3** : Éditeur de dessin fonctionnel.
- **Semaine 5** : Galerie et interactions sociales implémentées.
- **Semaine 7** : Profils utilisateurs et système de suivi terminés.
- **Semaine 8** : Tests terminés, application déployée.

- **Semaine 9** : Présentation prête pour le jury.

7. Contraintes et hypothèses

7.1 Contraintes

- Budget : Utilisation de Firebase (plan gratuit ou payant selon les besoins).
- Délais : Projet à finaliser dans un délai de 8 à 9 semaines.
- Ressources : Équipe de 1 à 3 développeurs (selon votre équipe réelle).

7.2 Hypothèses

- Les utilisateurs ont un compte Google ou une adresse email pour s'inscrire.
- Les utilisateurs disposent d'une connexion internet stable.
- Firebase reste disponible et performant pendant toute la durée du projet.

8.1 Budget

- Firebase : Plan gratuit (Spark) pour le développement, passage au plan Blaze si nécessaire (pay-as-you-go).
- Hébergement : Firebase Hosting (gratuit pour les besoins de base).
- Outils : VS Code (gratuit), GitHub (gratuit), Firebase CLI (gratuit).

8.2 Ressources matérielles

- Ordinateur avec Node.js 18+ installé.
- Connexion internet pour le développement et le déploiement.

10. Critères d'acceptation

10.1 Fonctionnalités

- L'utilisateur peut se connecter/s'inscrire et gérer son mot de passe.
- L'éditeur de dessin permet de créer, modifier, et sauvegarder des dessins.

- La galerie affiche les dessins avec tri, filtrage, et interactions (likes, partage).
- Les profils sont personnalisables (nom d'utilisateur, photo, bio) et incluent un système de suivi.
- L'application est responsive et fonctionne sur desktop et mobile.

10.2 Performance

- Temps de chargement des dessins < 2 secondes (sous bonne connexion).
- Pas de crash ou erreur bloquante lors de l'utilisation.

10.3 Sécurité

- Les données utilisateur sont protégées (Firestore, Storage).
- Les opérations sensibles (likes, commentaires, suivi) nécessitent une authentification.

11. Annexes

11.1 Maquettes (optionnel)

- Maquettes de l'éditeur de dessin, de la galerie, et de la page de profil (à fournir si disponibles).

11.2 Documentation technique

- Structure des collections Firestore (décrite dans la section 3.3).
- API des Firebase Functions (à documenter dans le README).

11.3 Références

- Documentation React : <https://react.dev/>
- Documentation Vite : <https://vite.dev/>
- Documentation Firebase : <https://firebase.google.com/docs>